

Step1_check_data.py

Step 1: confirm we are actually getting data back from the servo over the DPC-CAN, before trying anything else.

INCLUDES INITIALIZATION: when the DPC-CAN is plugged in fresh (without the Hitec app having been opened first), it sits in a dormant state and won't relay servo data. This script sends the same startup sequence the app wakes it up, then checks for live data.

Initialization sequence (reverse-engineered from app capture):

```
Phase 1: "probe"    - 02 53 00 ff 52 03 03    (x5)
Phase 2: "flush"    - 02 58 58 03              (x11)
Phase 3: "wake"     - 3a 41 3a 41 3a 41        (x1)
                    02 56 53 00 00 00 00 03    (x1)
                    -> DPC-CAN responds 04 22 18 (confirmed awake)
Phase 4: "activate" - 02 53 00 01 54 03 03    (x5)
                    02 53 01 01 55 03 03      (x5)
                    02 53 02 01 56 03 03      (x5)
                    02 53 03 01 57 03 03      (x5)
                    -> servo telemetry starts flowing after 03
```

The byte pattern for activate frames is 02 53 <N> 01 <N+0x54> 03 03 where N increments 00->01->02->03. Telemetry only starts after N=03.

Servo_control.py

Hitec MDB961WP-CAN servo control via DPC-CAN, pure pyserial.

Command an absolute ANGLE in degrees across the full travel (left/right).

=== Move frame (reverse-engineered) ===

02 42 00 f2 03 80 00 00 <LO> <HI> <SEQ> <CHK> 0a 03
byte 6 = 0x00 -> move flag
byte 7 = 0x00 -> constant
bytes 8-9 = position value, little-endian (byte8 low, byte9 high)
byte 10 = sequence counter (rolling)
byte 11 = checksum = sum(bytes[1:11]) & 0xFF

=== Position encoding (measured by hand) ===

The 16-bit value = MAGNITUDE (low 15 bits) + DIRECTION bit (bit 15): bit 15 (0x8000) clear -> RIGHT, set -> LEFT. Both sides use the same magnitude scale, mirrored.

Key facts established on the hardware:

- wire value 0 = true mechanical center
- magnitude 0 .. ~12000 = DEAD ZONE (no motion, sits at center)
- magnitude ~12500..15360 = active band, sweeps to full deflection
- magnitude 15360 = full deflection (~90 deg)
- magnitude 15361+ = REJECTED (servo ignores it) <-- hard ceiling

Travel is about +/-90 degrees (the app's +/-150 display was wrong).

=== Calibration (your measured % of travel, full = 90 deg) ===

value 12500 -> 10% -> 9.0 deg
value 13000 -> 15% -> 13.5 deg
value 13500 -> 20% -> 18.0 deg
value 14000 -> 30% -> 27.0 deg
value 14500 -> 50% -> 45.0 deg
value 15000 -> 70% -> 63.0 deg
value 15360 -> 100% -> 90.0 deg

Below ~12500 the servo is in the dead zone -> treated as center (0 deg). The relationship is non-linear, so we interpolate between points.

The servo holds position only while commanded, so we stream the target continuously in a background thread. Type an angle to retarget. 'q' quits.

SAFETY: range is about -90 to +90 deg. Keep the servo clear of obstructions. Low angles (near 0-9 deg) can be twitchy due to the steep step out of the dead zone.

Calib_collect_data.py

Option A - collect calibration points by hand.

Stream a raw value, hold it, observe the actual angle, write it down. We feed your verified (value, angle) pairs into the final controller, which interpolates between them. No theory about the encoding needed - we just map what the hardware actually does.

HOW TO USE:

1. Run this. It starts holding value 0 (center).
2. Type a raw value (e.g. 8000). Servo moves and HOLDS there.
3. Read the actual angle - from the servo physically, or reconnect the app briefly to read its angle display, then come back.
4. Write down: value -> angle
5. Repeat across the range. Aim for ~8-10 points spread out.
6. Give the list to Claude to build the final controller.

Suggested values to try (spread across the working range):

0, 4000, 8000, 11000, 12000, 13000, 14000, 15000, 15300 and for the other side: -4000, -8000, -12000, -15000

'q' to quit.

SAFETY: keep the servo clear; some jumps move fast.

Calib_raw_data_prober.py

Calibration prober: stream a RAW 16-bit value (no scale assumptions) and observe where the servo physically parks, to build the real value -> position mapping.

We confirmed verbatim frames move the servo, but the raw 16-bit value does NOT map to position the way we assumed (a frame decoding to -0.886 moved RIGHT, not left). So we stop assuming and measure directly.

You enter a raw integer (decimal, e.g. 0, 8000, 16000, -16000, 30000) or hex with 0x. It builds a valid frame (correct checksum) carrying that value in bytes 8-9 and streams it continuously so the servo actually holds. You watch where the horn points and record it.

Goal: enter several values and note physical position for each, so we can work out the true formula. Suggested sweep below in the prompt.

SAFETY: keep the servo clear. Type 'q' to quit (streaming stops).

Calib_with_reedback.py

Calibration tool WITH live position read-back. Streams a raw 16-bit value you type (into bytes 8-9 of the move frame) AND continuously reads the servo's reported position from its telemetry, printing both. This replaces eyeballing the horn with a real number.

The point: we keep mis-theorizing the value->position relationship. This tool measures it directly. You type a value, you SEE what position the servo reports reaching. We build a true table from that.

The telemetry frame looks like:

```
04 15 f3 03 88 <p0> <p1> 0b <p2> <p3> 00 00 <ctr> <chk> 0c 05
```

We print several candidate position fields so we can see which one tracks your commanded value. Once we know which field is real, we lock it in.

Type a raw integer (decimal or 0x hex). 'q' to quit.

SAFETY: keep the servo clear. Some values may move it far/fast.

Diag_relay_capture.py

Diagnostic: replay VERBATIM frames captured from the app during a real slider movement - no reconstruction, no math, exact bytes off the wire.

Purpose: settle whether our move frames are encoded wrong, or whether the problem is something about getting bytes onto the bus at all (e.g. the DPC-CAN needing the app's session first).

- If these real captured frames MOVE the servo -> our encoding has a specific bug and we compare to find it.
- If even these real frames do NOTHING -> the issue isn't the frame content; it's transport/session, and we look there instead.

We stream each captured frame continuously for a few seconds (the app streamed continuously, so we match that), pausing between them. Watch the servo during each phase and note any movement.

SAFETY: keep the servo clear - these command real positions across a wide range. Ctrl+C stops everything.

Step1b_register_sweep.py

Step 1b: confirm the servo is actually responding to OUR specific register read requests (not just background telemetry).

We send read requests for a few different registers, one at a time, with a quiet gap before/after each, and print everything that comes back. If a frame containing that exact register byte shows up right after each request, that confirms we've found the real response.

Step3_move_to_angle.py

Step 3 (final): command the servo to an absolute ANGLE in degrees.

KEY FACT we established: on the wire, the position value is a SIGNED number centered at 0 (0 = center, positive = right, negative = left). The counts shown in the Hitec app (8192 = center) are the app's own internal display scale, offset by 8192 from the wire value: $\text{wire_value} = \text{displayed_count} - 8192$

Calibration (from the app's Left / Center / Right readouts, converted to wire values):

-150.01 deg -> wire -6827

0.00 deg -> wire 0

+149.99 deg -> wire +6826

=> slope is 45.51 counts per degree, centered at 0.

Move frame:

02 42 00 f2 03 80 00 00 <LO> <HI> <SEQ> <CHK> 0a 03

byte 6 = 0x00 -> move flag

byte 7 = 0x00 -> constant

bytes 8-9 = signed 16-bit wire value, little-endian

byte 10 = sequence counter (rolling)

byte 11 = checksum = $\text{sum}(\text{bytes}[1:11]) \& 0xFF$

The servo holds position only while commanded, so we stream the target continuously in a background thread. Type an angle to retarget. 'q' quits.

SAFETY: travel is about -150 to +150 deg; values are clamped. Big retargets move at full speed
- keep the servo clear.